

# Requirements Elicitation and Validation: A Comprehensive Guide

Effective requirements gathering and validation are critical to project success: indeed, “two common reasons [projects] fail are lack of understanding of business needs and unclear requirements” <sup>1</sup>. This guide covers key *elicitation methods* (interviews, workshops, observation, document analysis), *requirement qualities* (clear, atomic, testable, etc.), *validation techniques* (reviews, prototyping, acceptance criteria) and *confirmation practices* (stakeholder sign-off, alignment checklists). Where helpful we provide example interview questions, Gherkin-style acceptance criteria, and a stakeholder-alignment checklist. All examples use simple, cross-industry scenarios (e.g. order-to-cash, onboarding, incident response, approval processes) so they apply broadly.

## Elicitation Methods

No single elicitation technique fits every context <sup>2</sup>, so analysts use a mix of methods suited to stakeholders and constraints. Four common methods are **interviews**, **workshops**, **observation**, and **document analysis**. Each has different strengths and ideal uses:

- **Interviews** (one-on-one or small group) involve asking open-ended questions to Subject Matter Experts (SMEs) and users <sup>3</sup>. This method digs deeply into stakeholder knowledge: “[by] exploring someone’s knowledge and needs in-depth...you ensure you understand the real, not just perceived, need” <sup>4</sup>. Interviews can be structured (with a fixed questionnaire) or unstructured (freeform). Analysts typically use open-ended “5 W” questions – *Who*, *What*, *When*, *Where*, *Why* – to elicit details. For example: “**Who** participates in this process and **what** are their roles?”, “**What** are the main steps in your [order-processing] workflow?”, “**Why** is this task performed this way?”, “**What issues or exceptions** do you face?” <sup>3</sup>. Effective interviewers balance structure and flexibility: they come prepared with key questions but remain ready to follow interesting tangents <sup>3</sup>. (See *Interview Question Bank* below for more sample questions.)
- **Requirements Workshops** (facilitated group sessions or JAD) bring together multiple stakeholders and experts in a short, focused meeting <sup>5</sup>. In a workshop “you ask everyone to sit down and hammer out the requirements together” over a few hours or days <sup>6</sup>. This intensive, collaborative format is ideal when you need to **align stakeholders quickly** or clarify conflicting needs. A well-run workshop (with a strong facilitator and note-taker) can elicit many ideas and resolve discrepancies on the spot. Success factors include inviting *all* key participants (too many can slow things; too few can miss requirements <sup>7</sup> <sup>8</sup>), setting a clear agenda, and using group activities or diagrams to spark discussion. For example, a workshop to define an *employee onboarding* process might include HR managers, IT staff, and trainers mapping out the current steps and brainstorming improvements. Workshops effectively combine the input of diverse SMEs, but they require careful planning (e.g. pre-read of current process) and strong facilitation <sup>7</sup>.

- **Observation (Shadowing)** has the analyst *watch* users perform their real work (often called contextual inquiry). This is useful when processes are poorly documented or when users may not fully articulate what they do. By observing tasks “you figure out exactly where users are at the start of your project” <sup>9</sup>. Observation can uncover tacit knowledge and workarounds that interviews miss. For instance, an analyst might spend time in a call center listening in on support calls, or follow a warehouse clerk picking items for an order. Caution: people often change their behavior when watched (“Hawthorne effect” <sup>10</sup>), so it’s important to explain that you are there to learn, not to judge, and to build rapport with users.
- **Document Analysis** means reviewing any existing materials – user manuals, process flows, system documentation, logs, regulations, competitor docs, etc. It is usually one of the first steps. Analyzing documentation helps identify **baseline requirements** and language. For example, reviewing a current invoice form or an old software requirements list can reveal needed fields or business rules. As one source notes, analysts “look at user guides, previous requirements, and existing systems” for clues <sup>11</sup>. Document analysis also includes *interface analysis* (examining screens or APIs) and reviewing data (reports, logs) to understand workflows. This method is efficient when up-to-date documentation exists, and it helps avoid reinventing the wheel. However, be aware that stale or incomplete docs can mislead; always confirm assumptions with people.
- **Other techniques:** Methods like surveys/questionnaires, focus groups, prototyping, or brainstorming are also used in practice. For example, questionnaires can reach many stakeholders asynchronously (though they lack follow-ups <sup>12</sup>), and focus groups are like workshops with more open discussion. Prototyping (building mock-ups) and JAD sessions may come later, but they too blur into the validation phase. We focus here on the four above, which cover most elicitation needs in a business context.

## Interview Question Bank

Below are sample questions that analysts can adapt when interviewing stakeholders. These are **open-ended** to encourage detailed responses. (Use them as a starting point; tailor language to your domain.)

- **Process and Workflow:**
  - “Can you walk me through the steps of [your process] from start to finish?”
  - “Who initiates this process, and who are the participants at each step?”
  - “What decisions or approvals are required, and at what points?”
- **Roles and Responsibilities:**
  - “What is your role in this process? Who else is involved?”
  - “How do different teams or departments hand off work to each other?”
- **Inputs and Outputs (Data):**
  - “What documents or data do you use? Where does this information come from?”
  - “What report or output do you generate? Where does it go next?”
- **Tools and Systems:**
  - “What software or tools do you currently use for this task?”
  - “If you had to describe your system’s screen/process, what does it look like?”
- **Rules and Exceptions:**
  - “What business rules or policies affect this process?”
  - “What exceptions or error conditions occur, and how are they handled?”

- **Pain Points and Risks:**

- “What are the biggest challenges or bottlenecks you face?”
- “Have you ever experienced a failure or outage in this process? What happened?”

- **Goals and Improvements:**

- “What is the purpose or goal of this process? How do you measure success?”
- “If you could change one thing about the current process/system, what would it be?”
- “Are there new requirements coming (regulations, expansion, etc.) that we should plan for?”

As [3] advises, use the **five Ws** throughout interviews and keep questions open-ended <sup>3</sup>. Follow up on any surprising detail, and let the interviewee explain freely, while noting any terms or steps that need later clarification. Always confirm **why** (“Why do you do it this way?”) to uncover real goals.

## Requirement Types and Qualities

A *requirement* is a statement of a needed capability, condition, or quality for a solution. Requirements can be **functional** (specific behaviors or processes) or **non-functional** (performance, security, usability, etc.). Good requirements share certain qualities – often summarized in BA and systems engineering literature. Key attributes include:

- **Clear (Unambiguous):** Each requirement should have only one interpretation. Use precise language. (E.g. instead of “fast”, say “< 2 seconds response time”.) Ambiguity leads to errors later.
- **Atomic (Singular):** A requirement should address one piece of functionality or criterion. Avoid compound requirements (“and/or”). The INCOSE standards call this *Singular* <sup>13</sup> or *Atomic* <sup>14</sup>. Splitting compound statements keeps requirements focused and testable.
- **Testable (Verifiable):** It must be possible to verify (via demonstration or measurement) whether the requirement is met. That means no subjective terms (“user-friendly”) without definition. If we can write a test or acceptance check for it, it’s testable <sup>15</sup> <sup>16</sup>.
- **Concise:** No unnecessary words or redundancy. A concise requirement avoids filler (“substantially”, “in some cases”) and states only what’s needed.
- **Complete:** The requirement covers all aspects of the need as agreed. (If it’s a list of steps, ensure none are omitted.) A set of requirements should cover all expected features and use-cases.
- **Consistent:** Requirements should not conflict with each other. Terminology should be uniform (e.g. use the same term for a “customer” throughout).
- **Feasible:** It should be realistic given the project’s scope, technology, schedule and budget. If a requirement seems impossible to implement or conflicts with constraints, flag it early.
- **Valuable (Necessary):** Each requirement should trace to a business need or stakeholder objective. Avoid “nice-to-have” features unless prioritized appropriately. Every requirement should deliver some value.
- **Traceable:** You should be able to link each requirement to its source (stakeholder goal, business rule, higher-level need) and to design or test artifacts. Traceability supports managing changes and verifying coverage.
- **Design-Free:** Requirements describe *what* is needed, not *how* to do it (that’s for design). For example, say “The system shall display order history to customers,” not “The system shall use a blue table to show order history.”

These qualities align with industry standards (BABOK, IEEE, INCOSE, etc.). For example, BABOK lists atomic, complete, consistent, concise, feasible, unambiguous, testable, prioritized, understandable among its desiderata <sup>14</sup> <sup>13</sup>. The INCOSE Guide similarly emphasizes *Unambiguous*, *Singular*, *Verifiable*, etc. <sup>13</sup>.

| Quality                 | Meaning/Example                                                                                                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Atomic/Singular         | The requirement contains only one idea. E.g. "System shall allow users to upload a file" (atomic) vs. "System shall allow uploads and edits of files" (compound). <sup>14</sup> |
| Clear/<br>Unambiguous   | Wording cannot be interpreted in multiple ways. Define terms if needed. E.g. specify units, formats.                                                                            |
| Testable/<br>Verifiable | There is a clear pass/fail criterion or measurement. E.g. "System shall send confirmation email <b>within 1 minute</b> of order completion."                                    |
| Complete                | No key detail is missing. E.g. if listing steps, include every step.                                                                                                            |
| Consistent              | Uses the same names and style. No internal contradictions.                                                                                                                      |
| Feasible                | Realistic given resources. ("Shall generate a million reports per second" might not be feasible.)                                                                               |
| Valuable                | Tied to a business need or stakeholder value, not a solution detail.                                                                                                            |
| Traceable               | Source of requirement is documented (e.g. stakeholder X said they need Y) and it links to tests/design.                                                                         |

*Atomic/testable* are crucial for writing acceptance criteria (see next section). Ambiguity or compound requirements often lead to rework. If you spot them (e.g. "reduce system lag" – how much?), clarify immediately. In practice, analysts review requirements iteratively: readability, completeness and testability are checked during walkthroughs or by specialized tools <sup>17</sup> <sup>13</sup> .

## Validation Techniques

Once requirements are captured, validation ensures they **truly reflect stakeholder needs and are ready to build**. Validation is an ongoing collaborative process, not a one-time task <sup>18</sup> <sup>19</sup> . Key validation techniques include:

- **Reviews and Walkthroughs:** Systematically examine requirements with stakeholders. This can be informal (walking a stakeholder through the draft) or formal (peer review session). Reviews help catch ambiguities, omissions, or irrelevant items early. For example, a business review meeting might have the analyst present the requirements document page by page to the sponsor, soliciting feedback. As one author notes, even during verification (checking grammar/format) analysts often find validation issues (wrong solution focus) <sup>20</sup> <sup>21</sup> . Formal review methods (inspections or walkthroughs) may use checklists to ensure each requirement meets quality criteria. Rework is much cheaper on a whiteboard than after coding, so invest time here.
- **Prototyping:** Build a quick model or mock-up of the system (UI screens, workflow simulation, or even paper sketches) to validate requirements with users. Prototypes let stakeholders *see* an early design and react: "Users usually don't know what they want until they see something they don't want" <sup>22</sup> . For example, create a simple clickable demo of a new invoice screen and confirm that stakeholders can perform all needed tasks with it. Prototypes are especially useful for new or complex applications (e.g. new order-entry system) where textual requirements might not convey the look-and-feel. The feedback from prototyping often uncovers hidden requirements or incorrect

assumptions. Modern low-code or UI tools make it faster to iterate prototypes. Remember, prototypes should be thrown away (or evolved); their purpose is elicitation/validation, not final implementation.

- **Acceptance Criteria and Test Cases:** Define clear *acceptance criteria* for each requirement or user story. Acceptance criteria are **specific, testable conditions** that a feature must satisfy for stakeholders to consider it done <sup>23</sup>. They act as a contract: developers and testers build and verify the feature against them, and stakeholders confirm them during acceptance testing. Well-written criteria greatly aid validation, because each item can be checked off. For example, for a “reset password” feature, criteria might include: *“Given a valid user on the login page, when they enter their email and click ‘Reset’, then the system emails a reset link within 2 minutes”*. (See **Acceptance Criteria Patterns** below for more on this.) By making requirements testable up front, acceptance criteria bridge elicitation and testing, ensuring nothing is overlooked in verification.
- **User Acceptance Testing (UAT):** Especially in predictive (waterfall) or hybrid projects, a UAT phase confirms the *built solution* meets requirements. During UAT, actual users execute real scenarios (often derived from requirements and acceptance criteria) on the completed system. This is the ultimate validation: if users approve, the requirements were correct and met. To prepare, analysts often help write test scripts based on requirements. Ideally, acceptance criteria already cover these tests. Any discrepancies found in UAT (e.g. a missing feature or misunderstanding) send us back to revisit requirements.
- **Traceability and Proving Fit:** Traceability matrices link requirements to design artifacts, code modules, and test cases. While not a “technique” like interviewing, establishing traceability ensures that every requirement is implemented and verified. As one guide notes, “Traceability tools...ensure alignment between project objectives and deliverables” <sup>24</sup>. During reviews or sign-off, analysts can walk stakeholders through these links to confirm coverage: e.g. showing how Requirement #42 (order approval must go to manager if > \$10K) appears in the design, test plan, and finally in the deployed system.

In short, use a combination of **static** techniques (reviews, walkthroughs, checklists) and **dynamic** techniques (prototypes, tests) to validate. One practitioner emphasizes that validation should start early and repeat often – ideally as soon as any requirements are drafted <sup>18</sup>. Every question an analyst asks about a requirement (why, how is it tested) is part of validation. The goal is that by the time of formal sign-off, stakeholders feel confident the documented requirements *and acceptance criteria* accurately capture their needs.

## Acceptance Criteria Formats (Gherkin-Style)

Acceptance criteria give requirements a concrete form. The two most common formats are **scenario-oriented (Given/When/Then)** and **rule-oriented (checklist)**. The Gherkin-style scenario format is widely used in Agile/BDD teams because it reads like a clear test case:

```
Scenario: [Descriptive name of the situation]
  Given [initial context or state]
  When [action taken by the user or system]
```

```
Then [expected outcome]
And [additional expected outcomes if needed]
```

Each line is plain English. For example, for a password-reset user story one might write <sup>25</sup> :

```
Scenario: Forgot password
  Given the user is on the login page
  When the user selects "Forgot password" and enters a valid email
  Then the system sends a password reset link to that email
  And the link expires after 24 hours
```

This clearly spells out what to test. The Altexsoft blog highlights this structure: "Given [precondition], When [action], Then [expected result]" <sup>26</sup> <sup>25</sup> .

Below is an illustrative example for a common cross-industry process (online order checkout):

```
Scenario: Completing checkout successfully
  Given the user has items in their shopping cart
  And the user is logged in
  When the user enters valid payment and shipping information
  Then the order is submitted and a confirmation email is sent
  And the order appears in the user's order history
```

Alternatively, **rule-oriented (checklist)** acceptance criteria are written as bullet points of conditions. For example, for the same checkout feature: - Payment form accepts Visa, MasterCard, or PayPal. - Billing address must be within supported countries (US, CA, UK). - Order total (items + tax + shipping) is correctly calculated. - Confirmation email includes order number and expected delivery date. This style is concise but less narrative. Both formats ensure requirements are measurable. In practice, use whatever style fits the team – but always capture explicit, testable conditions.

For ease of use, analysts often maintain templates or tables of acceptance criteria. The **Given/When/Then** pattern above is especially useful for critical flows and scenarios. It converts vague requirements into clear pass/fail tests, which streamlines validation. As one guide puts it, "Acceptance criteria are where ambiguity goes to die" <sup>27</sup> .

## Confirmation Practices

After validation activities, the final step is **confirming and approving** the requirements so the project can proceed. This typically means getting stakeholder agreement (formal or informal) and setting the requirements as a baseline. Key practices include:

- **Stakeholder Reviews and Sign-off:** Present the final requirements (and acceptance criteria) to all key stakeholders for approval. In traditional projects this may be a formal sign-off document; in Agile it may be the Product Owner's acceptance. Sign-off signifies that stakeholders *agree and commit* to

the documented requirements <sup>28</sup>. It's critical that stakeholders truly understand what they're approving. One authority warns against "requirements sign-off without understanding" – analysts should review the contents with stakeholders, address any concerns, and ensure commitment <sup>29</sup>. For example, have the process owner (e.g. Sales VP for order-to-cash) review each requirement that affects their domain.

- **Communication of Commitments:** Emphasize what sign-off means: stakeholders are agreeing to scope (both in terms of included requirements *and* the implicit exclusions). Some projects create a one-page summary of major commitments for executive alignment. Clear documentation of agreed requirements helps prevent "they didn't promise that!" later.
- **User Acceptance and Demonstrations:** During development, invite stakeholders to demos of increments (e.g. sprint reviews or UAT sessions). This serves as ongoing confirmation that the built solution matches their needs. If a demo reveals gaps, it may lead to new requirements or clarification – which should then go through the change-control process.
- **Traceability and Audit Trail:** Maintain a Requirements Traceability Matrix (or tool) so you can show how each requirement was approved. Link requirements to stakeholder sign-off notes, design docs, or test cases. This audit trail provides confidence that everything was covered.
- **Change Control:** Once requirements are baselined, any changes should follow a controlled process (impact analysis, review, re-approval). This confirms that all parties accept the change. Requiring re-sign-off on major changes ensures alignment remains intact.
- **Stakeholder Alignment Checklist:** To stay aligned throughout the project, use a checklist such as:
  - Identify **all** key stakeholders early and document their roles/concerns.
  - Map stakeholders (influence, interest) to prioritize engagement effort <sup>30</sup>.
  - Elicit and document each stakeholder's needs and expectations <sup>31</sup>.
  - Confirm that project goals and stakeholder goals are aligned (or identify trade-offs) <sup>32</sup>.
  - Establish a communication plan (channels, frequency, content) to keep stakeholders informed <sup>33</sup>.
  - Involve stakeholders in reviews, demos, or decision meetings to build ownership <sup>34</sup>.
  - Obtain explicit approval (sign-off, UAT acceptance, or Product Owner acceptance) on requirements before freezing scope <sup>28</sup>.
  - Monitor feedback and revisit alignment as needed ("never finished" <sup>35</sup>).

Citing some points: A recent article outlines a 7-step alignment process, including identifying and prioritizing stakeholders, uncovering their needs, aligning goals, and maintaining communication <sup>36</sup> <sup>32</sup>. Ultimately, requirement confirmation is successful when stakeholders "agree with and approve" the documented needs <sup>28</sup> and later see them fulfilled in the solution.

Putting it all together, thorough elicitation plus diligent validation & confirmation ensures the final requirements truly match stakeholder needs. By using a variety of elicitation techniques, adhering to requirement-quality attributes (clear, atomic, testable, etc.), and applying structured validation (reviews, prototypes, acceptance tests), analysts can reduce rework and miscommunication. Regular stakeholder engagement and clear approval processes (including an alignment checklist) keep everyone on the same

page. This combined approach – **comprehensive elicitation, rigorous validation, and careful confirmation** – is essential for delivering successful, value-aligned projects.

**Sources:** Industry-standard guides and articles on business analysis and requirements (e.g. BABOK, INCOSE, IEEE) and practitioner resources <sup>37</sup> <sup>14</sup> <sup>38</sup> <sup>23</sup> <sup>28</sup> . In particular, ModernAnalyst and AdaptiveUS blogs offer practical summaries of elicitation and validation best practices <sup>39</sup> <sup>38</sup> ; scopeMaster and INCOSE provide widely-cited quality criteria for requirements <sup>14</sup> <sup>13</sup> ; and contemporary Agile sources detail acceptance criteria patterns <sup>23</sup> <sup>25</sup> .

---

<sup>1</sup> 55 Business Analyst Requirements Gathering Interview Questions and Answers | Requirement

<https://www.requirement.com/business-analyst-requirements-gathering-interview-questions-answers/>

<sup>2</sup> <sup>3</sup> <sup>4</sup> <sup>5</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup> <sup>11</sup> <sup>12</sup> <sup>22</sup> <sup>37</sup> <sup>39</sup> The Top Five Go-To Requirements Elicitation Methods

<https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/2483/The-Top-Five-Go-To-Requirements-Elicitation-Methods.aspx>

<sup>13</sup> The Ultimate Guide to Requirements Management

<https://specinnovations.com/blog/guide-to-requirements-management>

<sup>14</sup> <sup>15</sup> Requirements quality attributes examined

<https://www.scopemaster.com/blog/requirements-quality-attributes/>

<sup>16</sup> <sup>17</sup> <sup>19</sup> <sup>24</sup> <sup>38</sup> Understanding Requirements Validation in Business Analysis

<https://www.adaptiveus.com/blog/requirements-validation/>

<sup>18</sup> <sup>20</sup> <sup>21</sup> How to Validate Requirements (BABOK 6.6)

<https://www.bridging-the-gap.com/validate-requirements-babok-6-6/>

<sup>23</sup> <sup>27</sup> 19 Acceptance Criteria Examples for Different Products, Formats and Scenarios

<https://www.prodpad.com/blog/acceptance-criteria-examples/>

<sup>25</sup> <sup>26</sup> Acceptance Criteria: Purposes, Types, Examples and Best Prac

<https://www.altexsoft.com/blog/acceptance-criteria-purposes-formats-and-best-practices/>

<sup>28</sup> <sup>29</sup> 3 Quick Tips for Requirements Sign-off — Business Analyst Learnings

<https://www.businessanalystlearnings.com/blog/2014/3/6/3-quick-tips-for-obtaining-requirements-sign-off>

<sup>30</sup> <sup>31</sup> <sup>32</sup> <sup>33</sup> <sup>34</sup> <sup>35</sup> <sup>36</sup> 7 Steps to Build Stakeholder Alignment

<https://simplystakeholders.com/stakeholder-alignment/>